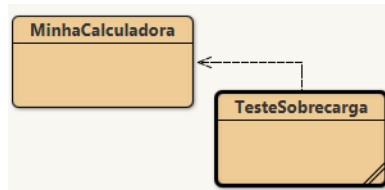


Sobrecarga de Métodos

A sobrecarga de métodos em Java permite que você defina múltiplos métodos com o mesmo nome, mas com parâmetros diferentes. O compilador Java escolherá automaticamente o método correto com base nos argumentos passados. Exemplo da classe MinhaCalculadora e sua classe teste:



```
public class MinhaCalculadora {

    public int soma(int num1, int num2){
        return num1 + num2;
    }

    public int soma(int num1, int num2, int num3){
        return num1 + num2 + num3;
    }

    public double soma(double num1, double num2){
        return num1 + num2;
    }

}
```

Nesse exemplo, temos três métodos soma com o mesmo nome, mas com diferentes tipos e/ou números de parâmetros. Isso é possível porque a assinatura dos métodos (nome e lista de parâmetros) é diferente em cada caso. Ao chamar o método soma, o compilador Java determinará qual versão do método deve ser chamada com base nos argumentos fornecidos. Por exemplo:

```
public class TesteSobrecarga {
    public static void main(String[] args) {
        MinhaCalculadora calc = new MinhaCalculadora();

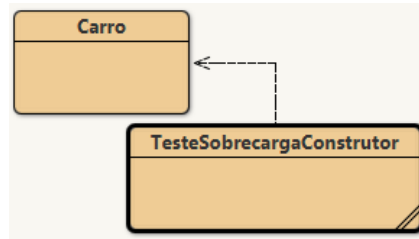
        int resultado1 = calc.soma(1, 2);           // Chama soma(int, int)
        int resultado2 = calc.soma(1, 2, 3);         // Chama soma(int, int, int)
        double resultado3 = calc.soma(1.5, 2.5);     // Chama soma(double, double)

        System.out.println("Resultado 1: " + resultado1);
        System.out.println("Resultado 2: " + resultado2);
        System.out.println("Resultado 3: " + resultado3);
    }
}
```

Então, na sobrecarga de métodos em Java. Você pode ter métodos com o mesmo nome, mas com diferentes tipos e/ou números de parâmetros, permitindo uma maior flexibilidade ao projetar suas classes.

Sobrecarga de Construtores

A sobrecarga de construtores em Java permite que você crie vários construtores com diferentes números ou tipos de parâmetros. Isso é útil quando você deseja fornecer várias maneiras de inicializar um objeto.



```
public class Carro {
    private String marca;
    private String modelo;
    private int numPassageiros;
    private double capCombustivel;
    private double consumoCombustivel;

    public Carro(String marca, String modelo, int numPassageiros, double capCombustivel, double consumoCombustivel) {
        this.marca = marca;
        this.modelo = modelo;
        this.numPassageiros = numPassageiros;
        this.capCombustivel = capCombustivel;
        this.consumoCombustivel = consumoCombustivel;
    }

    public Carro() {
    }

    public Carro(String marca, String modelo, int numPassageiros) {
        this.marca = marca;
        this.modelo = modelo;
        this.numPassageiros = numPassageiros;
        System.out.println("Chamando o construtor com 3 parâmetros");
    }

    public Carro(String marca, String modelo) {
        this(marca, modelo, 10);
        System.out.println("Chamando o construtor com 2 parâmetros");
    }
}
```

Neste exemplo, a classe Carro possui quatro construtores sobrecarregados:

O primeiro construtor recebe todos os atributos do carro.

O segundo construtor é um construtor vazio.

O terceiro construtor recebe a marca, modelo e número de passageiros, e inicializa esses atributos.

O quarto construtor recebe a marca e o modelo, e chama o terceiro construtor passando 10 como número padrão de passageiros.

No método main, você pode ver como criar instâncias da classe Carro utilizando os diferentes construtores disponíveis.

```
public class TesteSobrecargaConstrutor
{
    public static void main(String[] args) {
        Carro carro1 = new Carro("Marca1", "Modelo1", 4, 60.0, 15.0);
        Carro carro2 = new Carro();
        Carro carro3 = new Carro("Marca2", "Modelo2", 5);
        Carro carro4 = new Carro("Marca3", "Modelo3");
    }
}
```